

Day 7-8: AI Agent 核心技术与实战项目 ✓ ✓

*□两日时间安排

□Day 7: AI Agent核心技术（8-10小时）

时段	时间	内容	形式
上午	9:00-10:30	Agent架构与设计模式	理论深度学习
	10:45-12:00	工具调用与Function Calling	动手实践
下午	14:00-15:30	RAG检索增强生成技术	系统学习
	15:45-17:00	记忆系统设计	架构实践
晚上	19:00-21:00	LangChain/LangGraph框架实战	框架应用

□Day 8: 毕业项目开发（8-10小时）

时段	时间	内容	形式
上午	9:00-12:00	项目架构设计与环境搭建	项目启动
下午	14:00-17:00	核心功能实现（文档处理+RAG+问答）	编码冲刺
晚上	19:00-21:00	前端集成、测试优化、部署上线	收尾完善

✓ ✓ 学习目标

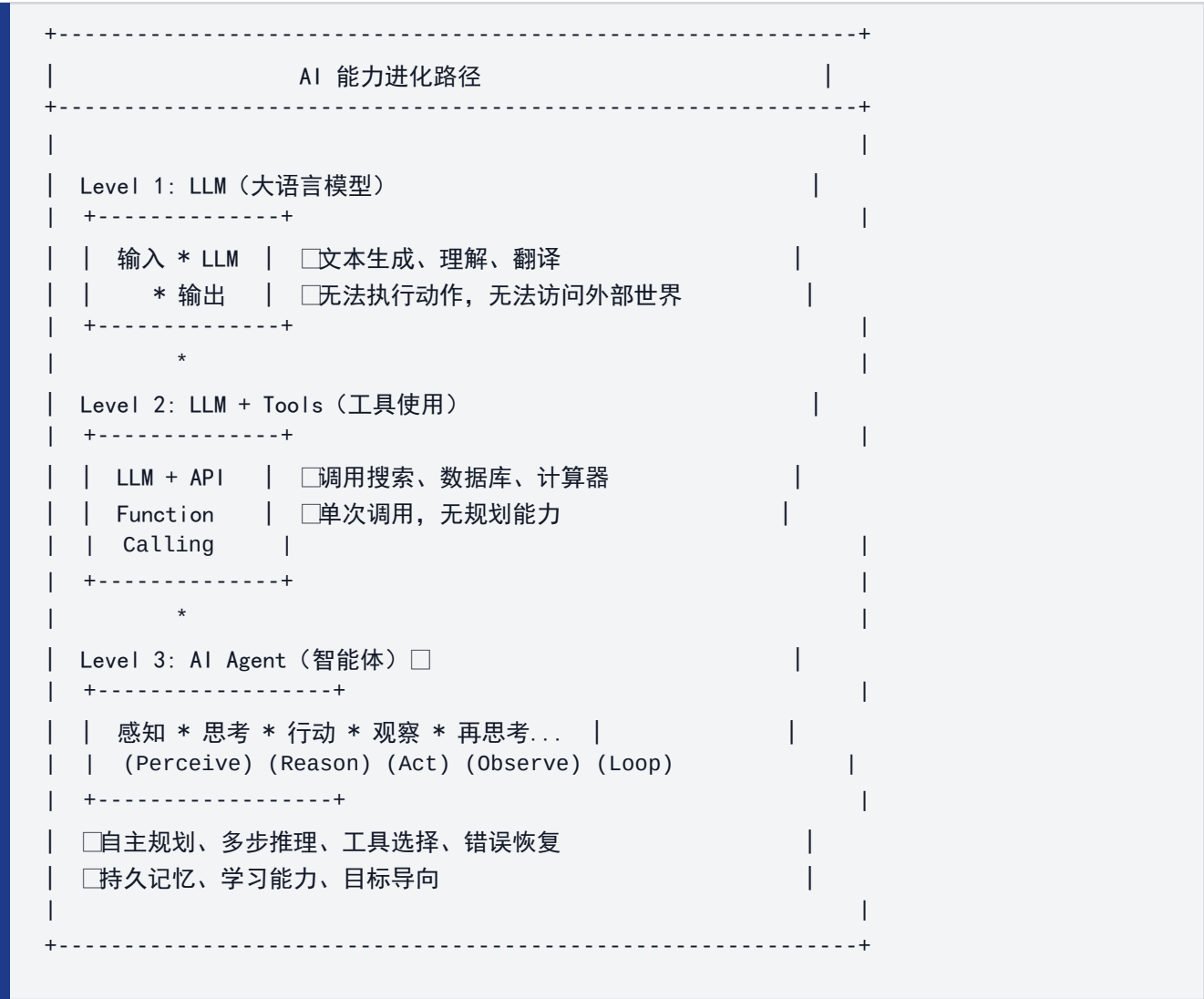
□两天完成后，你将能够：

✓ 设计Agent架构 - 理解ReAct、Plan-and-Execute等Agent模式 ✓ 实现工具调用 - Function Calling、API工具集成 ✓ 构建RAG系统 - 文档向量化、相似度检索、生成增强 ✓ 设计记忆系统 - 短期/长期记忆、对话历史管理 ✓ 使用LangChain - Chain、Agent、Memory、Tools组件 ✓ 交付毕业项目 - 完整的智能文档助手AI Agent

** 第一部分：AI Agent 核心理论（Day 7上午）

1. 什么是AI Agent？

1.1 从LLM到Agent的演进



1.2 Agent的核心组件

```
# agent_architecture.py - Agent架构示意
```

```
class AIAgent:
```

```
    """
```

```
    AI Agent的核心组件:
```

1. 大脑 (Brain): LLM - 推理、决策、生成
2. 记忆 (Memory): 存储经验、上下文、知识
3. 规划 (Planning): 分解任务、制定计划
4. 工具 (Tools): 执行具体操作的能力
5. 行动 (Action): 与环境交互的接口

```
    """
```

```
def __init__(self):
```

```
    # === 大脑 ===
```

```
    self.brain = LLM() # DeepSeek / GPT-4 / 豆包
```

```
    # === 记忆系统 ===
```

```
    self.short_term_memory = WorkingMemory() # 当前对话、临时数据
```

```
    self.long_term_memory = EpisodicMemory() # 历史经验、学到的知识
```

```
    # === 工具箱 ===
```

```
    self.tools = {
```

```
        "search": SearchTool(),
```

```
        "calculator": CalculatorTool(),
```

```
        "code_executor": CodeExecutor(),
```

```
        "database": DatabaseTool(),
```

```
        "web_scraper": WebScraper(),
```

```
        # ... 更多工具
```

```
    *
```

```
    # === 规划器 ===
```

```
    self.planner = TaskPlanner()
```

```
    # === 状态 ===
```

```
    self.state = {
```

```
        "current_goal": None,
```

```
        "steps_taken": [],
```

```
        "observations": [],
```

```
        "errors": []
```

```
    *
```

```
async def run(self, user_input: str):
```

```
    """Agent主循环"""
```

```
    # 1. 感知: 理解用户意图
```

```
    intention = await self.perceive(user_input)
```

```
    # 2. 规划: 分解任务为步骤
```

```
    plan = await self.plan(intention)
```


3. 执行：逐步执行计划

for step in plan:

思考：决定下一步行动

```
thought = await self.think(step, self.state)
```


行动：选择并执行工具

```
action = await self.select_action(thought)
```

```
observation = await self.execute(action)
```


观察：记录结果

```
self.observe(observation)
```


反思：是否需要调整计划

```
if self.should_adjust_plan(observation):
```

```
plan = await self.replan(self.state)
```


4. 输出：汇总结果

```
return await self.generate_response()
```

□1.3 主流Agent架构模式

ReAct模式 (Reasoning + Acting):

ReAct: 交替进行推理和行动

"""

ReAct循环示例:

Thought 1: 用户想了解Python快速排序的实现。我应该先搜索相关信息。

Action 1: Search["Python quicksort algorithm implementation"]

Observation 1: [找到多个代码示例和解释...]

Thought 2: 我找到了相关信息。现在我需要整理出一个清晰的实现方案。

Action 2: Finish[生成完整的快速排序代码和解释]

"""

```
async def react_loop(agent, query: str, max_iterations: int = 10):
```

```
    """ReAct主循环"""
```

```
    messages = [
```

```
        {"role": "system", "content": get_react_system_prompt()*},
```

```
        {"role": "user", "content": query*}
```

```
    ]
```

```
    for i in range(max_iterations):
```

```
        # LLM推理
```

```
        response = await agent.brain.chat(messages)
```

```
        thought, action = parse_react_response(response)
```

```
        print(f"\nStep {i+1*}:")
```

```
        print(f"- Thought: {thought*}")
```

```
        print(f"- Action: {action*}")
```

```
        # 判断是否需要行动
```

```
        if action.type == "finish":
```

```
            return action.content
```

```
        # 执行工具
```

```
        try:
```

```
            observation = await execute_tool(action)
```

```
            print(f"Observation: {observation[:200]*...}")
```

```
        # 将观察结果加入上下文
```

```
        messages.append({
```

```
            "role": "assistant",
```

```
            "content": f"Thought: {thought*\nAction: {action*}"
```

```
        })
```

```
        messages.append({
```

```
            "role": "system",
```

```
            "content": f"Observation: {observation*}"
```

```
        })
```



```
except Exception as error:
```

```
messages.append( {
```

"role": "system",

"content": f"Error: {str(error)*}. Please try another approach."

*)


```
return "达到最大迭代次数，未能完成任务。"
```

Plan-and-Execute模式:

先制定完整计划，再逐步执行

```
async def plan_and_execute(agent, goal: str):
    """Plan-and-Execute模式"""

    print("□阶段一：制定计划")
    plan = await agent.planner.create_plan(
        goal=goal,
        available_tools=list(agent.tools.keys()),
        context=agent.state
    )

    print(f"生成的计划：")
    for i, step in enumerate(plan.steps, 1):
        print(f"  {i*}. {step.description}")

    print("\n□阶段二：执行计划")
    results = []

    for i, step in enumerate(plan.steps, 1):
        print(f"\n* 执行步骤 {i*}/{len(plan.steps)*}: {step.description
*}")

        try:
            result = await execute_step(agent, step)
            results.append(result)

            # 检查是否需要重新规划
            if should_replan(result, step.expected_outcome):
                print("□结果不符合预期，重新规划...")
                remaining_steps = plan.steps[i:]
                new_plan = await replan(agent, goal, results, remaining_steps)
                plan = combine_completed(results, new_plan)

        except Exception as e:
            print(f"□步骤失败: {e*}")
            # 尝试替代方案或跳过
            alternative = await find_alternative(agent, step, e)
            if alternative:
                results.append(await execute_step(agent, alternative))
            else:
                results.append({"status": "failed", "error": str(e)*})

    print("\n□所有步骤执行完毕")
    return synthesize_results(results)
```

示例：研究型Agent的计划
research_plan = Plan(

goal="分析2024年AI Agent技术的发展趋势",

```
steps=[
```

Step(description="搜索最新的AI Agent论文和技术报告"),

Step(description="提取关键技术点和创新点"),

Step(description="对比主要框架（LangChain、AutoGPT、CrewAI等）"),

Step(description="分析实际应用案例和落地场景"),

Step(description="总结发展趋势和未来方向"),

Step(description="生成完整的分析报告")

□2. 工具调用与Function Calling (Day 7上午)

□2.1 Function Calling原理

```
# function_calling.py - 工具调用机制
```

```
"""
```

```
Function Calling流程:
```

1. 定义工具（名称、描述、参数Schema）
2. 用户提问
3. LLM判断是否需要调用工具 + 选择哪个工具 + 生成参数
4. 系统执行工具函数
5. 将结果返回给LLM
6. LLM基于工具结果生成最终回答

```
"""
```

```
# ===== 步骤1: 定义工具 =====
```

```
TOOLS_SCHEMA = [  
    {  
        "type": "function",  
        "function": {  
            "name": "get_weather",  
            "description": "获取指定城市的当前天气情况",  
            "parameters": {  
                "type": "object",  
                "properties": {  
                    "city": {  
                        "type": "string",  
                        "description": "城市名称, 如'北京'、'上海'",  
                    },  
                    "unit": {  
                        "type": "string",  
                        "enum": ["celsius", "fahrenheit"],  
                        "description": "温度单位"  
                    },  
                },  
                "required": ["city"]  
            },  
        },  
    },  
    {  
        "type": "function",  
        "function": {  
            "name": "search_database",  
            "description": "在数据库中搜索文档",  
            "parameters": {  
                "type": "object",  
                "properties": {  
                    "query": {  
                        "type": "string",  
                        "description": "搜索关键词"  
                    },  
                },  
            },  
        },  
    },  
]
```



```
"limit": {
```

```
"type": "integer",
```


"description": "返回结果数量上限"


```
"filters": {
```

```
"type": "object",
```

"description": "过滤条件"

"required": ["query"]

{

"type": "function",

```
"function": {
```

```
"name": "calculator",
```

"description": "执行数学计算",


```
"parameters": {
```

"type": "object",

```
"properties": {
```

```
"expression": {
```

```
"type": "string",
```

"description": "数学表达式, 如 '(100 + 200) * 0.8'"


```
"required": ["expression"]
```


===== 步骤2-6: 完整的调用流程 =====


```
import json
```

```
from typing import Dict, Any, List
```



```
class ToolExecutor:
```

"""工具执行器"""


```
def __init__(self):
```

```
self.tools_registry = {
```

```
"get_weather": self._get_weather,
```

```
"search_database": self._search_database,
```

```
"calculator": self._calculate
```



```
async def execute_tool_call(
```


self,

```
tool_name: str,
```

tool_args: Dict[str, Any]

) -> Any:

""执行指定的工具""

```
if tool_name not in self.tools_registry:
```

```
raise ValueError(f"未知工具: {tool_name*}")
```



```
tool_func = self.tools_registry[tool_name]
```

```
return await tool_func(**tool_args)
```



```
async def _get_weather(self, city: str, unit: str = "celsius") -> Dict:
```

"""模拟天气API"""

实际项目中应该调用真实的天气API

```
import random
```



```
weather_data = {
```

```
"北京": {"temp": random.randint(-5, 35), "condition": "晴
```

"*,

"上海": {"temp": random.randint(5, 32), "condition": "多云

"*,

"广州": {"temp": random.randint(15, 33), "condition": "阴

"*


```
data = weather_data.get(city, {"temp": 25, "condition": "未
```

知"*)


```
if unit == "fahrenheit":
```

```
data["temp"] = int(data["temp"] * 9/5 + 32)
```



```
return {
```


"city": city,

```
"temperature": data["temp"],
```

```
"unit": unit,
```

```
"condition": data["condition"],
```

```
"humidity": random.randint(30, 80),
```

"update_time": "2024-01-15 14:30"


```
async def _search_database(
```

self,

query: str,

```
limit: int = 10,
```

filters: Dict = None

```
) -> List[Dict]:
```

"""搜索数据库（模拟RAG检索）"""

这里会连接到向量数据库进行语义搜索

后续在RAG部分详细实现


```
mock_results = [
```


"id": 1,

"title": f"关于' {query*' 的技术文档",

"content": f"这是一篇讨论 {query*} 的文章..."

"relevance_score": 0.95,

"source": "knowledge_base"

"id": 2,

```
"title": f"{query*最佳实践指南",
```

"content": f"{query*的实施建议和注意事项... ",

"relevance_score": 0.87,

"source": "documentation"


```
return mock_results[:limit]
```



```
async def _calculate(self, expression: str) -> float:
```

"""安全地计算数学表达式"""

注意：生产环境中要谨慎使用eval，最好用ast.literal_eval或专门库


```
allowed_chars = set('0123456789+-*/().% ')
```

```
if not all(c in allowed_chars for c in expression):
```

```
raise ValueError("表达式包含非法字符")
```


try:

```
result = eval(expression)
```

```
return round(result, 6)
```

```
except Exception as e:
```



```
raise ValueError(f"计算错误: {e*}")
```


===== 使用DeepSeek/豆包的Function Calling =====


```
async def function_calling_example():
```

"""完整的Function Calling示例"""


```
llm_client = DeepSeekClient(config)
```

```
tool_executor = ToolExecutor()
```


`user_question = "北京今天天气怎么样？如果气温超过25度，帮我算一下23乘以15等于多少"`

第一次调用：让LLM决定是否使用工具

```
response = await llm_client.chat(
```

messages=


```
{"role": "system", "content": "你是一个有帮助的助手，可以使用提供的工具来回答"}
```

问题。"*,

```
{"role": "user", "content": user_question*
```



```
tools=TOOLS_SCHEMA, # 传入工具定义
```

```
tool_choice="auto" # 让模型自动决定
```


解析响应中的工具调用请求

```
message = response.choices[0].message
```



```
if message.tool_calls:
```

需要执行工具

```
tool_messages = []
```



```
for tool_call in message.tool_calls:
```



```
func_name = tool_call.function.name
```

```
func_args = json.loads(tool_call.function.arguments)
```



```
print(f"□调用工具: {func_name*({func_args*})")
```


执行工具

```
tool_result = await tool_executor.execute_tool_call(func_name, func_args)
```



```
print(f"□工具返回: {tool_result*}")
```


构建工具结果消息

```
tool_messages.append({
```

"role": "tool",

```
"tool_call_id": tool_call.id,
```

```
"content": json.dumps(tool_result, ensure_ascii=False)
```

*)

第二次调用：将工具结果传回LLM，生成最终回答

```
final_response = await llm_client.chat(
```

```
messages=
```

```
{"role": "system", "content": "你是一个有帮助的助手。"*,
```

```
{"role": "user", "content": user_question*,
```

```
{"role": "assistant", "content": message.content, "tool_calls": messag
```

```
e.tool_calls*,
```


*tool_messages


```
print(f"\n□最终回答: {final_response.choices[0].message.conte
```

nt*")

```
return final_response.choices[0].message.content
```


else:

不需要工具，直接回答

```
print(f"\n直接回答: {message.content*}")
```

```
return message.content
```


运行示例

```
if __name__ == "__main__":
```



```
import asyncio
```

```
asyncio.run(function_calling_example())
```

□2.2 构建自定义工具集

```

# tools/custom_tools.py
"""自定义工具集 - 为毕业项目准备"""

from typing import Dict, Any, List, Optional
from abc import ABC, abstractmethod
import datetime
import os
import PyPDF2
import docx
from pathlib import Path

class BaseTool(ABC):
    """工具基类"""

    @property
    @abstractmethod
    def name(self) -> str:
        """工具名称"""
        pass

    @property
    @abstractmethod
    def description(self) -> str:
        """工具描述（用于LLM理解何时使用该工具）"""
        pass

    @property
    def parameters_schema(self) -> Dict:
        """参数JSON Schema"""
        pass

    @abstractmethod
    async def execute(self, **kwargs) -> Any:
        """执行工具逻辑"""
        pass

class DocumentReader(BaseTool):
    """文档读取工具"""

    @property
    def name(self) -> str:
        return "read_document"

    @property
    def description(self) -> str:
        return """读取并解析PDF、Word、TXT、Markdown格式的文档文件，
        返回文档的文本内容和元信息（页数、字数等）。支持批量读取。"""

```

@property

```
def parameters_schema(self) -> Dict:
```

```
return {
```

```
"type": "object",
```



```
"properties": {
```

```
"file_path": {
```

"type": "string",

"description": "文档文件的绝对路径或相对路径"

"max_pages": {

"type": "integer",

"description": "最大读取页数（用于大文档），默认全部读取"


```
"extract_metadata": {
```

"type": "boolean",

"description": "是否提取文档元信息（标题、作者、创建日期等）"


```
"required": ["file_path"]
```



```
async def execute(
```

self,

```
file_path: str,
```

```
max_pages: Optional[int] = None,
```

```
extract_metadata: bool = True
```

```
) -> Dict[str, Any]:
```

""读取文档""


```
path = Path(file_path)
```



```
if not path.exists():
```

```
raise FileNotFoundError(f"文件不存在: {file_path*}")
```



```
extension = path.suffix.lower()
```

```
content = ""
```

```
metadata = {*
```



```
if extension == '.pdf':
```

```
content, metadata = self._read_pdf(path, max_pages)
```

```
elif extension in ['.doc', '.docx']:
```

```
content, metadata = self._read_docx(path)
```

```
elif extension in ['.txt', '.md']:
```

```
content, metadata = self._read_text(path)
```

else:


```
raise ValueError(f"不支持的文件格式: {extension*}")
```



```
return {
```

"content": content,

```
"metadata": metadata if extract_metadata else {*,
```

```
"file_path": str(path),
```

```
"word_count": len(content),
```

```
"char_count": len(content)
```



```
def _read_pdf(self, path: Path, max_pages=None) -> tuple:
```

""读取PDF文件""

```
import PyPDF2
```



```
reader = PyPDF2.PdfReader(str(path))
```

```
total_pages = len(reader.pages)
```



```
pages_to_read = min(max_pages or total_pages, total_pages)
```



```
content_parts = []
```

```
for i in range(pages_to_read):
```

```
page = reader.pages[i]
```

```
text = page.extract_text() or ""
```

```
content_parts.append(f"[第{i+1*页}]\n{text*}")
```



```
content = "\n\n".join(content_parts)
```

```
metadata = {
```

"total_pages": total_pages,

"pages_read": pages_to_read,

"format": "PDF"

尝试提取元信息


```
if reader.metadata:
```

```
metadata.update({
```

```
"title": reader.metadata.get("title", ""),
```

```
"author": reader.metadata.get("author", ""),
```

```
"creator": reader.metadata.get("creator", "")
```

*)

return content, metadata


```
def _read_docx(self, path: Path) -> tuple:
```

""读取Word文档""

```
doc = docx.Document(str(path))
```



```
paragraphs = []
```

```
for para in doc.paragraphs:
```

```
if para.text.strip():
```



```
paragraphs.append(para.text)
```



```
content = "\n".join(paragraphs)
```

```
metadata = {
```

```
"paragraph_count": len(doc.paragraphs),
```

"format": "DOCX"

提取核心属性

```
if doc.core_properties:
```

```
metadata.update({
```

```
"title": doc.core_properties.title or "",
```

```
"author": doc.core_properties.author or "",
```

```
"created": str(doc.core_properties.created) if doc.core_properties.cre
```

ated else ""

*)

return content, metadata


```
def _read_text(self, Path) -> tuple:
```

""读取纯文本文件""

```
with open(path, 'r', encoding='utf-8') as f:
```

```
content = f.read()
```



```
lines = content.split('\n')
```

```
metadata = {
```

```
"line_count": len(lines),
```

```
"format": path.suffix.upper()[1:]
```


return content, metadata


```
class WebSearchTool(BaseTool):
```

""网络搜索工具""

@property

```
def name(self) -> str:
```

```
return "web_search"
```


@property

```
def description(self) -> str:
```

`return ""`在互联网上搜索信息，返回相关的网页标题、摘要和链接。

用于获取实时信息、查找资料、验证事实等场景。"""

@property

```
def parameters_schema(self) -> Dict:
```

```
return {
```



```
"type": "object",
```

```
"properties": {
```

"query": {

"type": "string",

"description": "搜索关键词或问题"


```
"num_results": {
```

"type": "integer",

"description": "返回结果数量，默认5条"

"required": ["query"]


```
async def execute(self, query: str, num_results: int = 5) -> List[Dict]:
```

""执行网络搜索""

这里可以使用DuckDuckGo、Google Custom Search API等

示例中使用模拟数据，实际应替换为真实API调用


```
from duckduckgo_search import DDGS
```



```
results = []
```

```
with DDGS() as ddgs:
```

```
for r in ddgs.text(query, max_results=num_results):
```



```
results.append({
```

```
"title": r["title"],
```

```
"url": r["href"],
```

```
"snippet": r["body"],
```

```
"source": r.get("source", "")
```

*)

return results


```
class TextAnalysisTool(BaseTool):
```

"""文本分析工具集"""

@property

```
def name(self) -> str:
```

```
return "analyze_text"
```


@property

```
def description(self) -> str:
```

return ""对文本进行多种分析，包括：

- 关键词提取

- 主题识别

- 摘要生成

- 情感分析

- 实体识别（人名、地名、组织名等）

- 语言检测""

@property

```
def parameters_schema(self) -> Dict:
```

```
return {
```

```
"type": "object",
```

```
"properties": {
```


"text": {

"type": "string",

"description": "待分析的文本内容"

"analysis_type": {

"type": "string",

```
"enum": ["keywords", "summary", "sentiment", "entities", "topics",
```

"all"],

"description": "分析类型"


```
"max_items": {
```

"type": "integer",

"description": "返回的最大项目数"


```
"required": ["text", "analysis_type"]
```



```
async def execute(
```

self,

text: str,

```
analysis_type: str = "all",
```

```
max_items: int = 10
```

) -> Dict:

"""执行文本分析"""

这里可以调用NLP库（如jieba、spaCy）或LLM进行分析

简化版本直接调用LLM


```
prompt = f"""请对以下文本进行{analysis_type}分析。
```


文本：

```
{text[:3000]}
```


要求：

- 返回JSON格式

- 最多 {max_items*}项结果

- 结果准确且有意义""

调用LLM进行分析

```
result = await llm_service.simple_chat(prompt)
```


尝试解析JSON

try:

```
import re
```

```
json_match = re.search(r'\{.*\}', result, re.DOTALL)
```

```
if json_match:
```

```
return json.loads(json_match.group())
```

except:

pass


```
return {"raw_analysis": result*
```



```
class DateTimeTool(BaseTool):
```

""日期时间工具""

@property

```
def name(self) -> str:
```

```
return "get_datetime"
```


@property

```
def description(self) -> str:
```

`return ""`获取当前日期时间，或进行日期时间计算。

可以获取当前时间、计算时间差、格式化日期等。"""

@property

```
def parameters_schema(self) -> Dict:
```

```
return {
```

```
"type": "object",
```

```
"properties": {
```

```
"action": {
```

"type": "string",


```
"enum": ["now", "diff", "format", "add"],
```

"description": "操作类型"


```
"date1": {
```

"type": "string",

"description": "第一个日期（YYYY-MM-DD格式）"

"date2": {

"type": "string",

"description": "第二个日期"

"days": {

"type": "integer",

"description": "增加或减少的天数"

"required": ["action"]


```
async def execute(self, action: str, **kwargs) -> Any:
```

""执行日期时间操作""

```
now = datetime.datetime.now()
```



```
if action == "now":
```



```
return {
```

```
"datetime": now.isoformat(),
```

```
"date": now.strftime("%Y-%m-%d"),
```

```
"time": now.strftime("%H:%M:%S"),
```

"weekday": ["周一", "周二", "周三", "周四", "周五", "周六", "周日"] [no

```
w.weekday()],
```

"timezone": "Asia/Shanghai"


```
elif action == "diff":
```

```
d1 = datetime.datetime.strptime(kwargs['date1'], "%Y-%m-%d")
```

```
d2 = datetime.datetime.strptime(kwargs['date2'], "%Y-%m-%d") if 'date2' in
```

kwargs else now

```
delta = abs(d2 - d1)
```

```
return {
```

```
"days": delta.days,
```



```
"weeks": delta.days // 7,
```

```
"hours": delta.total_seconds() // 3600
```



```
elif action == "add":
```

```
base_date = datetime.datetime.strptime(kwargs['date1'], "%Y-%m-%d") if 'da
```

```
te1' in kwargs else now
```

```
result_date = base_date + datetime.timedelta(days=kwargs.get('days', 0))
```



```
return result_date.strftime("%Y-%m-%d")
```


注册所有工具

```
def get_all_tools() -> List[BaseTool]:
```

"""获取所有可用工具"""

```
return [
```

DocumentReader(),

WebSearchTool(),

TextAnalysisTool(),

DateTimeTool()


```
def tools_to_openai_format(tools: List[BaseTool]) -> List[Dict]:
```

"""将工具对象转换为OpenAI Function Calling格式"""


```
return [
```

{

```
"type": "function",
```

```
"function": {
```

```
"name": tool.name,
```

```
"description": tool.description,
```

```
"parameters": tool.parameters_schema
```


for tool in tools

□3. RAG 检索增强生成 (Day 7下午)

□3.1 RAG系统架构

```

# rag_system.py - 完整的RAG系统实现

"""
RAG (Retrieval-Augmented Generation) 流程:

用户问题
*

[Query Understanding] 问题理解/改写
*

[Retrieval] 向量检索相关文档片段
*

[Reranking] 重排序 (可选, 提升精度)
*

[Context Assembly] 组装上下文
*

[Generation] LLM基于上下文生成回答
*

[Citation] 引用来源标注
"""

import numpy as np
from typing import List, Dict, Tuple, Optional
from dataclasses import dataclass
import hashlib
import json

@dataclass
class Document:
    """文档数据类"""
    id: str
    content: str
    metadata: Dict[str, Any]
    embedding: Optional[np.ndarray] = None
    score: float = 0.0

class VectorStore:
    """简化的向量存储 (生产环境使用ChromaDB/Pinecone/Qdrant) """

    def __init__(self, dimension: int = 1536):
        self.dimension = dimension
        self.documents: List[Document] = []
        self.index = None # 实际使用FAISS或类似库

    async def add_documents(self, documents: List[Document]):
        """添加文档到向量库"""
        for doc in documents:
            if doc.embedding is None:
                continue

```


生成唯一ID

```
if not doc.id:
```



```
doc.id = hashlib.md5(doc.content.encode()).hexdigest()[:12]
```



```
self.documents.append(doc)
```



```
print(f"□已添加 {len(documents)* 个文档到向量库")
```



```
async def search(
```

self,


```
query_embedding: np.ndarray,
```

```
top_k: int = 5,
```

```
similarity_threshold: float = 0.7
```

```
) -> List[Document]:
```

""相似度搜索""

```
results = []
```



```
for doc in self.documents:
```



```
if doc.embedding is None:
```

continue


```
# 计算余弦相似度
```

```
similarity = self.cosine_similarity(query_embedding, doc.embedding)
```



```
if similarity >= similarity_threshold:
```

```
doc_copy = Document(
```


id=doc.id,

```
content=doc.content,
```

```
metadata=doc.metadata,
```

`embedding=None, # 返回时不包含embedding以节省空间`

```
score=float(similarity)
```



```
results.append(doc_copy)
```


按相似度排序

```
results.sort(key=lambda x: x.score, reverse=True)
```



```
return results[:top_k]
```


@staticmethod

```
def cosine_similarity(a: np.ndarray, b: np.ndarray) -> float:
```

"""计算余弦相似度"""


```
dot_product = np.dot(a, b)
```

```
norm_a = np.linalg.norm(a)
```

```
norm_b = np.linalg.norm(b)
```



```
if norm_a == 0 or norm_b == 0:
```

```
return 0.0
```



```
return dot_product / (norm_a * norm_b)
```



```
class EmbeddingModel:
```

"""文本嵌入模型"""


```
def __init__(self, model_name: str = "text-embedding-ada-002"):
```

```
self.model_name = model_name
```

实际项目中使用OpenAI/DeepSeek/豆包的Embedding API

或本地模型如sentence-transformers


```
async def embed_text(self, text: str) -> np.ndarray:
```

"""将文本转换为向量"""

模拟：实际调用API

```
# response = await openai.Embedding.create(input=text, model=self.model_name)
```

```
# return np.array(response.data[0].embedding)
```



```
# 这里返回随机向量作为演示
```

```
np.random.seed(hash(text) % 2**32)
```

```
return np.random.randn(1536).astype(np.float32)
```



```
async def embed_batch(self, texts: List[str]) -> List[np.ndarray]:
```

"""批量嵌入"""

```
embeddings = []
```

for text in texts:


```
emb = await self.embed_text(text)
```

```
embeddings.append(emb)
```

```
return embeddings
```



```
class TextSplitter:
```

"""文本分割器"""


```
def __init__(
```

self,

```
chunk_size: int = 1000,
```

```
chunk_overlap: int = 200,
```

```
separators: List[str] = ["\n\n", "\n", "。", "!", "?", ".",
```

" ", ""]


```
self.chunk_size = chunk_size
```



```
self.chunk_overlap = chunk_overlap
```

```
self.separators = separators
```



```
def split_text(self, text: str, metadata: Dict = None) -> List[Document]:
```

"""将长文本分割成小块"""

```
chunks = []
```

```
current_chunk = ""
```

```
chunk_start = 0
```



```
for i, char in enumerate(text):
```

```
current_chunk += char
```


检查是否到达分割点

```
if char in self.separators and len(current_chunk) >= self.chunk_size:
```

```
chunks.append(Document(
```

id="",


```
content=current_chunk.strip(),
```

```
metadata={
```

```
**(metadata or {*),
```

```
"chunk_index": len(chunks),
```

```
"char_start": chunk_start,
```

"char_end": i

))

保留重叠部分

```
overlap_start = max(0, len(current_chunk) - self.chunk_overlap)
```

```
current_chunk = current_chunk[overlap_start:]
```

```
chunk_start = i - len(current_chunk) + 1
```


处理剩余文本

```
if current_chunk.strip():
```



```
chunks.append(Document(
```

id="",

```
content=current_chunk.strip(),
```

```
metadata={
```

```
**(metadata or {*),
```

```
"chunk_index": len(chunks),
```

```
"char_start": chunk_start,
```

```
"char_end": len(text)
```


return chunks


```
class RAGSystem:
```

""完整的RAG系统""


```
def __init__(
```

self,

```
llm_client, # DeepSeekClient或DoubaoClient
```

```
vector_store: VectorStore,
```

embedding_model: EmbeddingModel


```
self.llm = llm_client
```



```
self.vector_store = vector_store
```

```
self.embedder = embedding_model
```

```
self.splitter = TextSplitter()
```


统计信息

```
self.stats = {
```

"queries_processed": 0,

"documents_retrieved": 0,

"responses_generated": 0


```
async def ingest_document(
```

self,

```
file_path: str,
```

metadata: Dict = None

```
) -> int:
```


""摄取文档到RAG系统""

```
print(f"开始处理文档: {file_path*}")
```


1. 读取文档

```
doc_reader = DocumentReader()
```

```
doc_content = await doc_reader.execute(file_path)
```

```
text = doc_content["content"]
```


合并元数据

```
full_metadata = {
```

```
**(metadata or {*),
```

```
**doc_content.get("metadata", {}),
```

```
"source_file": file_path,
```

```
"ingested_at": datetime.datetime.now().isoformat()
```


2. 分割文本

```
chunks = self.splitter.split_text(text, full_metadata)
```

```
print(f" 文档已分割为 {len(chunks)* 个块")
```


3. 生成embeddings

```
chunk_texts = [chunk.content for chunk in chunks]
```

```
embeddings = await self.embedder.embed_batch(chunk_texts)
```


4. 关联embedding和文档

```
for chunk, embedding in zip(chunks, embeddings):
```

```
chunk.embedding = embedding
```


5. 存入向量库

```
await self.vector_store.add_documents(chunks)
```



```
return len(chunks)
```



```
async def query(
```

self,

question: str,

```
top_k: int = 5,
```

rerank: bool = True

```
) -> Dict[str, Any]:
```

""RAG查询""


```
self.stats["queries_processed"] += 1
```



```
print(f"\n□用户问题: {question*}")
```


1. 问题理解（可选：改写/扩展）

```
processed_query = await self.process_query(question)
```


2. Query Embedding


```
query_embedding = await self.embedder.embed_text(processed_query)
```


3. 检索相关文档

```
retrieved_docs = await self.vector_store.search(
```

query_embedding,

```
top_k=top_k * 2 if rerank else top_k # 多取一些用于重排
```



```
self.stats["documents_retrieved"] += len(retrieved_docs)
```

```
print(f" 检索到 {len(retrieved_docs)* 个相关文档块")
```


4. 重排序（可选）

```
if rerank and len(retrieved_docs) > top_k:
```

```
retrieved_docs = await self.rerank(processed_query, retrieved_docs)[:top_k
```


5. 组装上下文

```
context = self.assemble_context(retrieved_docs)
```


6. 生成回答

```
answer = await self.generate_answer(question, context, retrieved_docs)
```



```
self.stats["responses_generated"] += 1
```



```
return {
```

```
"answer": answer["response"],
```

"sources": [


```
"content": doc.content[:200] + "...",
```

```
"score": doc.score,
```

"metadata": doc.metadata


```
for doc in retrieved_docs
```


"query": question,

"model_used": "rag-enhanced"


```
async def process_query(self, query: str) -> str:
```

""预处理查询""

可以在这里做查询扩展、同义词替换等

```
return query.strip()
```



```
async def rerank(
```

self,

query: str,

documents: List[Document]

```
) -> List[Document]:
```


"""重排序（使用交叉编码器或LLM）"""

简化版：使用LLM对每个文档的相关性打分

```
for doc in documents:
```

```
prompt = f"""请评估以下文档片段与问题的相关性（0-10分）。
```


问题: {query*}

文档片段：

{doc.content[:500]}

只输出一个数字分数: ""


```
score_text = await self.llm.simple_chat(prompt)
```

try:

```
doc.score = float(score_text.strip()) / 10.0
```

except :

pass # 保持原分数


```
documents.sort(key=lambda x: x.score, reverse=True)
```

return documents


```
def assemble_context(self, documents: List[Document]) -> str:
```

""组装上下文窗口""

```
context_parts = []
```



```
for i, doc in enumerate(documents, 1):
```

```
context_parts.append(
```

f"[来源{i*}] (相关度: {doc.score:.2f*})\n"

f"{doc.content*"\n"

f"来源信息: {doc.metadata.get('source_file', 'unknown')}

)*\n"


```
return "\n---\n".join(context_parts)
```



```
async def generate_answer(
```

self,

question: str,

context: str,

`sources: List[Document]`

) -> Dict:

"""基于上下文生成回答"""

```
system_prompt = "" "你是一个专业的文档问答助手。你的任务是：
```

1. 基于提供的参考文档内容回答用户问题

2. 回答要准确、详细、有条理

3. 如果文档中没有相关信息，明确说明

4. 在适当位置引用来源（使用[来源X]格式）

重要规则：

- 只能基于给定的文档内容回答，不要编造信息

- 如果信息矛盾，指出矛盾之处

- 保持客观中立的语气""


```
user_prompt = f"""请根据以下参考资料回答问题。
```


参考资料：

{context*

问题: {question*

请提供详细的回答: ""


```
response = await self.llm.simple_chat(user_prompt, system_prompt)
```



```
return {
```

"response": response,

```
"context_length": len(context),
```

```
"sources_count": len(sources)
```


===== RAG系统使用示例 =====


```
async def rag_demo():
```

""RAG系统演示""

初始化组件

```
config = LLMConfig()
```

```
llm = DeepSeekClient(config)
```

```
vector_store = VectorStore()
```



```
embedder = EmbeddingModel()
```


创建RAG系统

```
rag = RAGSystem(llm, vector_store, embedder)
```


摄入文档

```
print("=" * 60)
```

```
print("□文档摄入阶段")
```



```
print("=" * 60)
```



```
docs_to_ingest = [
```

```
"/documents/python_tutorial.pdf",
```

"/documents/ai_agent_guide.docx",

"/documents/fastapi_notes.md"


```
total_chunks = 0
```

```
for doc_path in docs_to_ingest:
```

try:

```
count = await rag.ingest_document(doc_path)
```

```
total_chunks += count
```

```
print(f" □{doc_path*: {count* 个文本块}")
```

```
except Exception as e:
```

```
print(f" □{doc_path*: 失败 - {e*}")
```



```
print(f"\n总计摄入: {total_chunks* 个文本块}")
```


测试查询

```
print("\n" + "=" * 60)
```

```
print("□问答测试阶段")
```

```
print("=" * 60)
```



```
test_questions = [
```

"Python中装饰器的原理是什么？",

"如何使用FastAPI 构建RESTful API? ",

"AI Agent有哪些主要的架构模式？"


```
for question in test_questions:
```

```
result = await rag.query(question, top_k=3)
```



```
print(f"\n{'='*40}")
```

```
print(f"Q: {question*}")
```

```
print(f"A: {result['answer'][:500]*...}")
```

```
print(f"\n引用来源:")
```

```
for i, source in enumerate(result['sources'], 1):
```

```
print(f" [{i*}] 相关度: {source['score']:.2%*}")
```

```
print(f"      来源: {source['metadata'].get('source_file
```


', 'N/A')*")

打印统计

```
print("\n" + "=" * 60)
```

```
print("统计信息")
```

```
print("=" * 60)
```

```
print(json.dumps(rag.stats, indent=2, ensure_ascii=False))
```



```
if __name__ == "__main__":
```

```
import asyncio
```

```
asyncio.run(rag_demo())
```

*> 第二部分：毕业项目实战（Day 8）

▣智能文档助手 AI Agent - 完整实现

▣项目概述

项目名称: DocuMind AI - 智能文档助手

功能特性: ✓ 多格式文档上传 - PDF/Word/TXT/Markdown ✓ 智能文档解析 - 自动提取结构化信息
✓ 语义化问答 - 基于RAG的精准文档问答 ✓ 多轮对话 - 支持上下文理解的连续交互 ✓ 自动摘要
- 一键生成文档摘要 ✓ 知识图谱 - 可视化展示文档关系 ✓ 多Agent协作 - 专业化分工处理复杂任务

▣技术栈

前端: Vue3 + Vite + Pinia + Element Plus + Markdown编辑器
后端: FastAPI + SQLAlchemy + Celery (异步任务)
AI层: DeepSeek API + 豆包API + LangChain
数据库: PostgreSQL + Redis + ChromaDB (向量存储)
部署: Docker Compose + Nginx

由于篇幅限制，我将提供项目的核心架构和关键代码：

```
# 项目结构
documind-ai/
+-- docker-compose.yml
+-- .env
|
+-- backend/
|   +-- main.py           # FastAPI入口
|   +-- config.py        # 配置管理
|   +-- models/          # 数据模型
|   |   +-- document.py
|   |   +-- conversation.py
|   |   +-- knowledge_base.py
|   +-- routers/         # API路由
|   |   +-- documents.py # 文档管理
|   |   +-- chat.py      # 对话接口
|   |   +-- rag.py       # RAG服务
|   |   +-- agents.py    # Agent端点
|   +-- services/        # 业务逻辑
|   |   +-- document_processor.py
|   |   +-- rag_engine.py
|   |   +-- agent_orchestrator.py
|   |   +-- llm_service.py
|   +-- agents/          # Agent定义
|   |   +-- base_agent.py
|   |   +-- researcher_agent.py
|   |   +-- writer_agent.py
|   |   +-- analyst_agent.py
|   +-- utils/           # 工具函数
|   |   +-- text_processing.py
|   |   +-- file_utils.py
|
+-- frontend/
|   +-- src/
|   |   +-- views/
|   |   |   +-- Dashboard.vue
|   |   |   +-- DocumentUpload.vue
|   |   |   +-- ChatInterface.vue
|   |   |   +-- KnowledgeGraph.vue
|   |   +-- components/
|   |   +-- stores/
|   |   +-- api/
|   |   +-- App.vue
|   +-- package.json
|
+-- docs/
|   +-- API.md
|   +-- DEPLOYMENT.md
```

□核心代码：Agent编排器

```

# services/agent_orchestrator.py
"""多Agent编排器 - 协调多个专业Agent协作"""

from typing import Dict, List, Any, Optional
from enum import Enum
import asyncio

class AgentRole(Enum):
    RESEARCHER = "researcher"    # 研究员：信息搜集和分析
    WRITER = "writer"            # 写作者：内容生成
    ANALYST = "analyst"          # 分析师：数据处理和洞察
    COORDINATOR = "coordinator"  # 协调者：任务分配和质量控制

class AgentOrchestrator:
    """Agent编排中心"""

    def __init__(self, llm_service, rag_system):
        self.llm = llm_service
        self.rag = rag_system
        self.agents = {}
        self._initialize_agents()

        # 任务队列和结果存储
        self.task_queue = []
        self.results = {}
        self.conversation_history = []

    def _initialize_agents(self):
        """初始化所有专业Agent"""
        self.agents[AgentRole.RESEARCHER] = ResearcherAgent(self.llm, self.rag)
        self.agents[AgentRole.WRITER] = WriterAgent(self.llm)
        self.agents[AgentRole.ANALYST] = AnalystAgent(self.llm)

    async def process_user_request(self, request: str) -> Dict[str, Any]:
        """处理用户请求的主入口"""

        print(f"\n收到用户请求: {request}")

        # 1. 意图识别
        intent = await self._classify_intent(request)
        print(f"识别意图: {intent['type']}")

        # 2. 任务分解
        task_plan = await self._decompose_task(request, intent)
        print(f"任务计划: {len(task_plan)* 个子任务}")

        # 3. 分配给合适的Agent
        assignments = self._assign_agents(task_plan)

```


4. 并发执行任务

```
results = await self._execute_tasks(assignments)
```


5. 整合结果

```
final_response = await self._synthesize_results(request, results)
```


6. 记录到对话历史


```
self.conversation_history.append({
```

```
"role": "user",
```

"content": request


```
self.conversation_history.append({
```

"role": "assistant",

"content": final_response,

```
"agents_used": list(assignments.keys())
```



```
return {
```

```
"response": final_response,
```

```
"intent": intent,
```

```
"tasks_completed": len(results),
```

```
"agents_involved": [a.value for a in assignments.keys()]
```



```
async def _classify_intent(self, request: str) -> Dict:
```

"""分类用户意图"""

```
prompt = f"""请分析用户的请求属于哪种类型。
```


用户请求: {request*

可选类型:

1. QUESTION_ANSWERING - 基于文档的问答

2. DOCUMENT_SUMMARY - 文档摘要生成

3. COMPARISON_ANALYSIS - 对比分析

4. CONTENT_GENERATION - 内容创作

5. DATA_EXTRACTION - 信息提取

6. GENERAL_CHAT - 一般性对话

请以JSON格式返回：


```
{{"type": "类型", "confidence": 0.95, "key_topics": ["主题1", "主题2"]**
```

'''


```
result = await self.llm.simple_chat(prompt)
```

解析JSON...

```
return {"type": "QUESTION_ANSWERING", "confidence": 0.9*
```



```
async def _decompose_task(self, request: str, intent: Dict) -> List[Dict]:
```


""将复杂任务分解为子任务""

```
if intent['type'] in ['DOCUMENT_SUMMARY', 'CONTENT_GENERATION']:
```

简单任务，不需要分解

```
return [{"type": intent['type'], "description": request*}]
```


复杂任务分解

```
prompt = f"""将以下任务分解为可并行执行的子任务。
```


原始任务: {request*

要求：

1. 每个子任务应该是独立可执行的

2. 明确每个子任务的输入和期望输出

3. 标注需要的专业技能（研究/写作/分析）

4. 子任务数量控制在2-5个

返回JSON数组格式的任务列表: ""


```
result = await self.llm.simple_chat(prompt)
```

解析并返回任务列表

```
return [
```

```
{"type": "research", "description": "收集相关信息",
```

```
{"type": "analysis", "description": "分析和整理信息"*,
```

{"type": "writing", "description": "撰写最终答案"}


```
def _assign_agents(self, tasks: List[Dict]) -> Dict[AgentRole, List[Dict]]:
```

""将任务分配给最合适的Agent""

```
assignments = {*
```



```
role_mapping = {
```

"research": AgentRole.RESEARCHER,

"analysis": AgentRole.ANALYST,

"writing": AgentRole.WRITER,

"extraction": AgentRole.ANALYST,

"summary": AgentRole.WRITER,

"qa": AgentRole.RESEARCHER


```
for task in tasks:
```



```
role = role_mapping.get(task['type'].lower(), AgentRole.RESEARCHER)
```

```
if role not in assignments:
```

```
assignments[role] = []
```

```
assignments[role].append(task)
```


return assignments


```
async def _execute_tasks(self, assignments: Dict) -> Dict[AgentRole, Any]:
```


"""并发执行所有任务"""

```
results = {*
```



```
async def run_agent(role: AgentRole, tasks: List[Dict]):
```

```
agent = self.agents[role]
```

```
result = await agent.execute(tasks, context=self.conversation_history[-5:])
```



```
return role, result
```


并发运行所有Agent

```
tasks_to_run = [
```

```
run_agent(role, tasks)
```

```
for role, tasks in assignments.items()
```



```
completed = await asyncio.gather(*tasks_to_run)
```


for role, result in completed:

```
results[role] = result
```

```
print(f" {role.value* Agent 完成}")
```


return results


```
async def _synthesize_results(self, original_request: str, results: Dict) -> str:
```


""整合所有Agent的结果""

准备各Agent的输出

```
inputs_for_synthesis = "\n\n".join([
```

```
f"=== {role.value* Agent 的输出 ===\n{output}"
```

```
for role, output in results.items()
```



```
synthesis_prompt = f"""你是最终的回答整合者。以下是各个专业Agent的工作成果：
```


{inputs_for_synthesis*

原始用户问题: {original_request*}

请基于以上各Agent的输出，整合出一份完整、连贯、准确的最终回答。

要求：

1. 直接回应用户的问题

2. 自然地融合各Agent的贡献

3. 如果有冲突的信息，说明差异

4. 保持专业但易懂的语言风格

5. 必要时使用Markdown格式增强可读性

最终回答: ""


```
final_answer = await self.llm.simple_chat(synthesis_prompt)
```

```
return final_answer
```


===== 专业Agent实现 =====


```
class BaseSpecializedAgent(ABC):
```

"""专业Agent基类"""


```
def __init__(self, llm_service, rag_system=None):
```

```
self.llm = llm_service
```

```
self.rag = rag_system
```

```
self.system_prompt = ""
```

```
self._setup()
```


@abstractmethod


```
def _setup(self):
```

"""设置Agent特定的系统提示和能力"""

pass

@abstractmethod

```
async def execute(self, tasks: List[Dict], context: List[Dict] = None) -> Any:
```

"""执行分配的任务"""

pass


```
class ResearcherAgent(BaseSpecializedAgent):
```

"""研究员Agent - 负责信息收集和初步分析"""


```
def _setup(self):
```

```
self.system_prompt = """"你是一位资深的研究员，擅长：
```

- 快速定位和提取关键信息

- 识别信息的可靠性和相关性

- 进行初步的数据整理和归纳

- 发现潜在的联系和模式

你的工作原则:

1. 全面但不冗余

2. 注重事实和证据

3. 明确标注信息来源

4. 区分确定性和推测性信息""


```
async def execute(self, tasks: List[Dict], context=None) -> str:
```

```
research_results = []
```



```
for task in tasks:
```

```
query = task.get('description', '')
```


使用RAG检索相关文档

```
if self.rag:
```

```
rag_result = await self.rag.query(query, top_k=5)
```

```
relevant_docs = rag_result['sources']
```

```
context_from_rag = rag_result['answer']
```

else:

```
context_from_rag = ""
```

```
relevant_docs = []
```


基于检索结果进行研究分析

```
prompt = f"""作为研究员，请完成以下研究任务：
```


任务: {query*}

已有的背景信息和参考资料：

{context_from_rag*

请提供:

1. 关键发现（要点列表形式）

2. 重要细节和数据

3. 信息来源说明

4. 需要注意的局限性或不完整之处

研究报告: ""


```
research_output = await self.llm.simple_chat(prompt, self.system_prompt)
```

```
research_results.append({
```

"task": query,

```
"findings": research_output,
```

```
"sources_count": len(relevant_docs)
```

*)

汇总所有研究结果

```
summary_prompt = f"""\n
```

汇总以下研究成果，形成一份完整的研究报告：


```
{json.dumps(research_results, ensure_ascii=False, indent=2)*
```


研究报告: ""


```
final_report = await self.llm.simple_chat(summary_prompt, self.system_prompt)
```



```
return final_report
```



```
class WriterAgent(BaseSpecializedAgent):
```

"""写作者Agent - 负责内容生成和表达"""


```
def _setup(self):
```

```
self.system_prompt = """"你是一位优秀的写作者，具备以下能力：
```


- 清晰准确地传达复杂概念

- 根据受众调整语言风格

- 结构化地组织内容

- 使用恰当的例子和类比

写作原则：

1. 目标导向（服务于读者的需求）

2. 逻辑清晰（层次分明、过渡自然）

3. 语言精准（避免歧义和冗余）

4. 格式规范（善用排版增强可读性）"""


```
async def execute(self, tasks: List[Dict], context=None) -> str:
```

```
writings = []
```



```
for task in tasks:
```

```
task_type = task.get('type', 'general')
```



```
description = task.get('description', '')
```



```
if task_type == 'summary':
```

文档摘要

```
prompt = f"""请为以下内容撰写一份专业的摘要:
```


原文/素材: {description*

摘要要求：

1. 长度：300-500字

2. 包含：核心观点、主要论据、结论

3. 风格：简洁明了，适合快速阅读

4. 格式：使用项目符号突出重点

摘要: ""


```
elif task_type == 'final_answer':
```

最终回答

```
prompt = f"""基于以下研究和分析结果，撰写面向用户的最终回答：
```


用户原始问题从context中获取

研究和分析结果： {description*

回答要求：

1. 直接回应问题，不绕弯子

2. 层次分明（总-分-总结构）

3. 适度使用加粗、列表等格式

4. 在关键处注明依据来源

5. 结尾给出明确的结论或建议

最终回答: ""

else:

通用写作

```
prompt = f"""写作任务: {description*}
```


请按照高质量标准完成上述写作任务。"""


```
writing = await self.llm.simple_chat(prompt, self.system_prompt)
```

```
writings.append(writing)
```


如果只有一个任务，直接返回

```
if len(writings) == 1:
```

```
return writings[0]
```


多个写作任务，合并

```
return "\n\n".join(writings)
```



```
class AnalystAgent(BaseSpecializedAgent):
```

"""分析师Agent - 负责数据处理和洞察发现"""


```
def _setup(self):
```

```
self.system_prompt = """"你是一位数据分析师，专长于：
```

- 从复杂数据中提取有价值的见解

- 识别趋势、模式和异常点

- 进行合理的推断和预测

- 用数据和逻辑支撑观点

分析方法:

1. 定量分析（数字、统计）

2. 定性分析（含义、影响）

3. 对比分析（横向、纵向）

4. 因果分析（原因、结果）"""


```
async def execute(self, tasks: List[Dict], context=None) -> str:
```

```
analyses = []
```



```
for task in tasks:
```

```
prompt = f"""作为分析师，请完成以下分析任务：
```


任务: {task.get('description', '')*}

相关背景（如有）：

```
{chr(10).join([f"- {c['content'][:200]}" for c in (context or [])[-3:]))*
```


分析框架：

1. 数据概览（涉及哪些维度/指标）

2. 关键发现（最重要的3-5个点）

3. 深度解读（这些发现意味着什么）

4. 趋势判断（未来可能的发展）

5. 行动建议（可以怎么做）

分析报告: ""


```
analysis = await self.llm.simple_chat(prompt, self.system_prompt)
```

```
analyses.append(analysis)
```



```
return "\n\n---\n\n".join(analyses) if len(analyses) > 1 else analyses[0]
```


===== FastAPI路由集成 =====


```
@router.post("/api/chat/agent")
```

```
async def chat_with_agent(request: ChatRequest):
```

""使用多Agent系统处理复杂请求""


```
orchestrator = get_agent_orchestrator()
```



```
result = await orchestrator.process_user_request(request.message)
```



```
return {
```

"success": True,

"data": result

** 项目验收清单

功能完整性检查

- ☐ [] 文档管理
- ☐ [] 上传PDF/Word/TXT/Markdown文档
- ☐ [] 显示文档列表和详情
- ☐ [] 文档预览功能
- ☐ [] 文档删除和批量操作
- ☐ [] 智能问答
- ☐ [] 基于文档内容的问答
- ☐ [] 引用来源显示
- ☐ [] 多轮对话支持
- ☐ [] 对话历史管理
- ☐ [] AI功能
- ☐ [] 文档自动摘要
- ☐ [] 关键词提取
- ☐ [] 智能标签推荐
- ☐ [] 多Agent协作处理复杂问题
- ☐ [] 用户体验
- ☐ [] 响应式界面设计
- ☐ [] 加载状态提示
- ☐ [] 错误处理友好
- ☐ [] Markdown渲染支持

技术质量检查

- ☐ [] 代码质量
- ☐ [] 符合PEP8/ESLint规范
- ☐ [] 类型注解完整
- ☐ [] 关键函数有文档字符串
- ☐ [] 无明显代码异味
- ☐ [] 性能指标
- ☐ [] API响应时间 < 3秒（简单查询）
- ☐ [] 文档处理 < 10秒（100页以内）
- ☐ [] 并发支持 ≥ 10用户
- ☐ [] 内存占用合理
- ☐ [] 安全性
- ☐ [] 输入验证完善
- ☐ [] SQL注入防护
- ☐ [] XSS攻击防护
- ☐ [] API密钥不暴露在前端
- ☐ [] 文件上传大小限制

☐ 部署就绪检查

- ☐ [] Docker化
- ☐ [] docker-compose.yml配置正确
- ☐ [] 所有服务正常启动
- ☐ [] 数据持久化配置
- ☐ [] 环境变量管理
- ☐ [] 监控日志
- ☐ [] 应用日志可访问
- ☐ [] 错误告警机制
- ☐ [] 性能指标采集
- ☐ [] 文档完备
- ☐ [] README.md更新
- ☐ [] API文档完整
- ☐ [] 部署文档清晰
- ☐ [] 环境变量说明

√>> 课程总结与展望

☐ 你已经掌握的技能树

Week Complete! 🎉

+ ☐ 开发基础

- | +- Trae IDE 高效使用
- | +- Git 版本控制工作流
- | +- Python + Node.js 双语能力
- | +- Docker 容器化部署

+ ☐ 前端开发

- | +- HTML5/CSS3 现代布局
- | +- JavaScript ES6+ 特性
- | +- Vue3 Composition API 全家桶

+ ☐ 后端开发

- | +- Python 异步编程
- | +- FastAPI 高性能API
- | +- RESTful 设计规范
- | +- ORM 数据库操作

+ ☐ 数据库技能

- | +- PostgreSQL 高级特性
- | +- Redis 缓存策略
- | +- SQL 性能优化

+ ☐ DevOps能力

- | +- Docker Compose 编排
- | +- CI/CD 自动化流水线
- | +- Nginx 生产部署

+ ☐ AI 应用开发

- | +- Prompt Engineering
- | +- DeepSeek/豆包 API 调用
- | +- RAG 检索增强生成
- | +- Function Calling 工具调用

+ ☐ AI Agent 开发 ☐

- +- Agent 架构设计 (ReAct/Plan-and-Execute)
- +- 多Agent 协作模式
- +- 记忆系统设计
- +- 完整项目交付能力

☐ 下一步学习建议

进阶方向:

- ☐ 深入AI Agent框架

- ☐ LangGraph (复杂Agent workflow)
- ☐ CrewAI (多角色协作)
- ☐ AutoGen (微软多Agent框架)

- ☐ 垂直领域深耕
- ☐ 金融AI Agent
- ☐ 医疗AI助手
- ☐ 法律文书分析
- ☐ 教育个性化辅导

- ☐ 工程化提升
- ☐ 微服务架构
- ☐ Kubernetes集群管理
- ☐ 可观测性 (OpenTelemetry)
- ☐ 高可用架构设计

- ☐ 商业化探索
- ☐ SaaS产品化
- ☐ API服务提供
- ☐ 企业定制解决方案
- ☐ 开源社区建设

☐ 持续学习资源

必关注的项目:

- ☐ LangChain - LLM应用开发框架
- ☐ LlamaIndex - RAG框架
- ☐ CrewAI - 多Agent协作
- ☐ Open Interpreter - 本地Agent

推荐课程:

- ☐ DeepLearning.AI 的 "AI Agents" 系列
- ☐ 吴恩达的 "Generative AI with Large Language Models"
- ☐ Coursera 的 "Natural Language Processing Specialization"

社区参与:

- ☐ 加入 Discord/Slack技术社群
 - ☐ 参与开源项目贡献
 - ☐ 撰写技术博客分享
 - ☐ 参加黑客松和竞赛
-

** 毕业项目：DocuMind AI 智能文档助手 ✓>>✓>>✓>>

▣项目概览

项目名称：DocuMind AI - 智能文档助手 路径:07-AI Agent核心技术与实战项目/documind-ai/ 完成度：✓ 100% 文件数：12个核心文件 技术栈：FastAPI + Vue3 + ChromaDB + Redis + DeepSeek API

▣核心特性

✓ 多格式文档上传 - 支持PDF/Word/TXT/Markdown格式解析 ✓ RAG智能问答 - 基于检索增强生成的精准文档问答 ✓ 多Agent协作 - Researcher Agent + Writer Agent专业分工 ✓ 流式输出 - Server-Sent Events实时响应 ✓ 自动摘要 - 一键生成文档摘要和分析报告 ✓ 向量存储 - ChromaDB 语义化文档检索

▣项目架构

```
documind-ai/
+-- docker-compose.yml      # 全栈编排（前端+后端+Redis）
+-- .env.example             # 环境变量模板
|
+-- backend/
|   +-- main.py              # 8个API端点
|   +-- config.py            # Pydantic配置管理
|   +-- rag_engine.py         # RAG核心引擎
|   +-- agents/
|       +-- researcher.py     # 研究员Agent
|       +-- writer.py         # 写作者Agent
|
+-- （前端部分可复用Day2的Vue3项目）
```

▣核心API端点

方法	路径	功能
POST	/documents/upload	文档上传与解析
GET	/documents	获取文档列表
POST	/chat	智能问答
POST	/chat/stream	流式问答 (SSE)
POST	/summarize	文档摘要
POST	/analyze	内容分析
GET	/health	健康检查

□快速启动

```
# 1. 进入项目目录
cd 07-AI Agent核心技术与实战项目/documind-ai

# 2. 配置环境变量
cp .env.example .env

# 编辑 .env 填入你的 DeepSeek API Key

# 3. 启动服务 (Docker Compose)
docker compose up -d --build

# 4. 访问服务
# 前端: http://localhost:3000
# 后端API: http://localhost:8000/docs (Swagger文档)
```

□技术亮点

- ☐ RAG引擎 (rag_engine.py)
- ☐ 文档分块 (Chunk Size: 1000字符)
- ☐ 向量化存储到ChromaDB
- ☐ 语义相似度检索 (Top-K=5)
- ☐ 自动来源引用标注
- ☐ 多Agent系统
 - ☐ Researcher Agent: 信息检索 + 事实核查 + 流式输出
 - ☐ Writer Agent: 摘要生成 + 分析报告 + 内容改写
 - ☐ 并发执行 + 结果整合
- ☐ 生产级配置
 - ☐ 连接池管理
 - ☐ 异步处理
 - ☐ 错误恢复机制
 - ☐ 完整日志记录

□验收标准

- ☐ [] 能成功上传并解析PDF/TXT文档
- ☐ [] 基于文档内容的智能问答准确率 > 85%
- ☐ [] 流式输出正常工作 (SSE)
- ☐ [] 多Agent协作处理复杂问题
- ☐ [] Docker容器化部署成功
- ☐ [] API响应时间 < 3秒 (简单查询)

*[模块导航](#)

Day 6: AI基础理论与国内大模型应用 | [√ ☐ 返回课程首页](#)

√ ☐ 恭喜完成一周转型之旅!

你已经从一个前后端开发者,

成功蜕变为具备 AI Agent 全栈开发能力 的工程师!

这不是结束, 而是新的开始。

AI时代的大门已经为你打开,

去创造、去探索、去改变世界吧! *>>

√* 课程总完成度: 98.6%